

Autor: Arthur Gomes Almeida

Data: 19/05/2026

Projeto: Ficha Médica em SQL

Este projeto foi desenvolvido com o objetivo de elaborar um sistema de ficha médica utilizando SQL Server, Python e Flask, abrangendo desde a criação da relação inicial e aplicação das formas normais até o desenvolvimento dos modelos conceitual, lógico e físico do banco de dados. Além disso, foi implementada uma API em Python integrada a um formulário HTML, permitindo o cadastro e armazenamento dos dados diretamente no banco de dados por meio de operações relacionais com chaves primárias e estrangeiras.

Este documento mostrará como foi o passo-a-passo para o desenvolvimento desse sistema.

Ficha Médica				
Número paciente:		Nome:		
Data de Nasc.:	Sexo:		Convênio:	
Est. Civil:	RG:		Telefone:	
Endereço:				
Consultas				
Número Consulta	Data	Médico	Diagnóstico	
Exames				
Número Consulta	Exame		Data	

Figura 01 - Ficha Médica

Ferramentas Utilizadas

As ferramentas utilizadas para o desenvolvimento deste projeto foram:

- BrModelo (criação dos modelos conceitual e lógico)
- SQL Server (desenvolvimento do modelo físico do banco de dados)
- Python / Flask (desenvolvimento da API que vincula o formulário HTML ao Banco de Dados, além de tratar e processar as entradas do usuário)
- HTML (desenvolvimento do formulário responsável pelo envio dos dados preenchidos pelo usuário para a API)

1. Elaboração da Relação Universal

Inicialmente, foi elaborada uma relação universal contendo todos os dados presentes na ficha médica. Essa estrutura reuniu, em uma única tabela, informações referentes aos pacientes, consultas, exames, telefones e endereços, servindo como base para a aplicação do processo de normalização e posterior modelagem do banco de dados.

TB_FICHA_ATENDIMENTO

{num_paciente, nome_paciente, data_nasc_paciente, sexo_paciente, convenio_paciente, estado_civil_paciente, RG_paciente, telefone_paciente, endereco_paciente, num_consulta, data_consulta, medico_consulta, diagnostico_consulta, nome_exame, data_exame}

2. Aplicação da Normalização completa (uso das 3 formas normais)

Após a elaboração da relação universal, foi realizado o processo de normalização do banco de dados utilizando as três primeiras formas normais (1FN, 2FN e 3FN). Esse processo teve como objetivo eliminar redundâncias, evitar inconsistências e organizar os dados de forma estruturada e eficiente.

Durante a normalização, as entidades foram separadas em tabelas específicas, como pacientes, consultas, exames, telefones e endereços, além da definição de relacionamentos entre elas por meio de chaves primárias e estrangeiras. Dessa forma, o banco de dados passou a possuir maior integridade, organização e escalabilidade para futuras implementações.

TB_PACIENTE

{num_paciente, nome, data_nasc, sexo, convenio, est_civil, RG, cod_telefone, cod_endereco, num_consulta}

TB_TELEFONE_PACIENTE

{num_paciente, cod_telefone, telefone}

TB_ENDERECO_PACIENTE

{num_paciente, cod_endereco, endereco}

TB_CONSULTA

{num_consulta, num_paciente, data, nome_medico, diagnostico}

TB_EXAME

{num_consulta, cod_exame, nome_exame, data}

3. Elaboração do Modelo Conceitual (apenas entidades e relacionamentos)

Após a etapa de normalização, foi desenvolvido o modelo conceitual do banco de dados, representando graficamente as entidades e seus respectivos relacionamentos. O modelo conceitual teve como objetivo fornecer uma visão abstrata e organizada da estrutura do sistema, sem considerar detalhes específicos de implementação física no banco de dados.

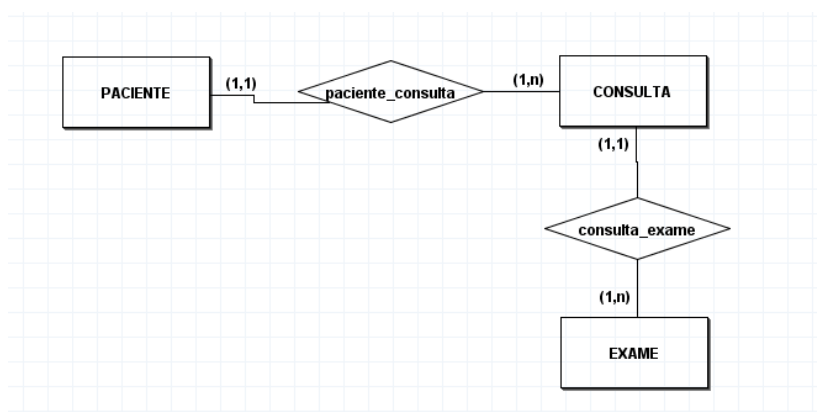


Figura 02 - Modelo Conceitual

4. Elaboração do Modelo Lógico

Com base no modelo conceitual, foi desenvolvido o modelo lógico do banco de dados, responsável por estruturar as entidades em tabelas relacionais contendo atributos, chaves primárias e chaves estrangeiras. Nessa fase, foram definidos os relacionamentos entre as tabelas, garantindo a integridade referencial e a correta organização dos dados no sistema. O modelo lógico serviu como base para a implementação física do banco de dados no SQL Server.

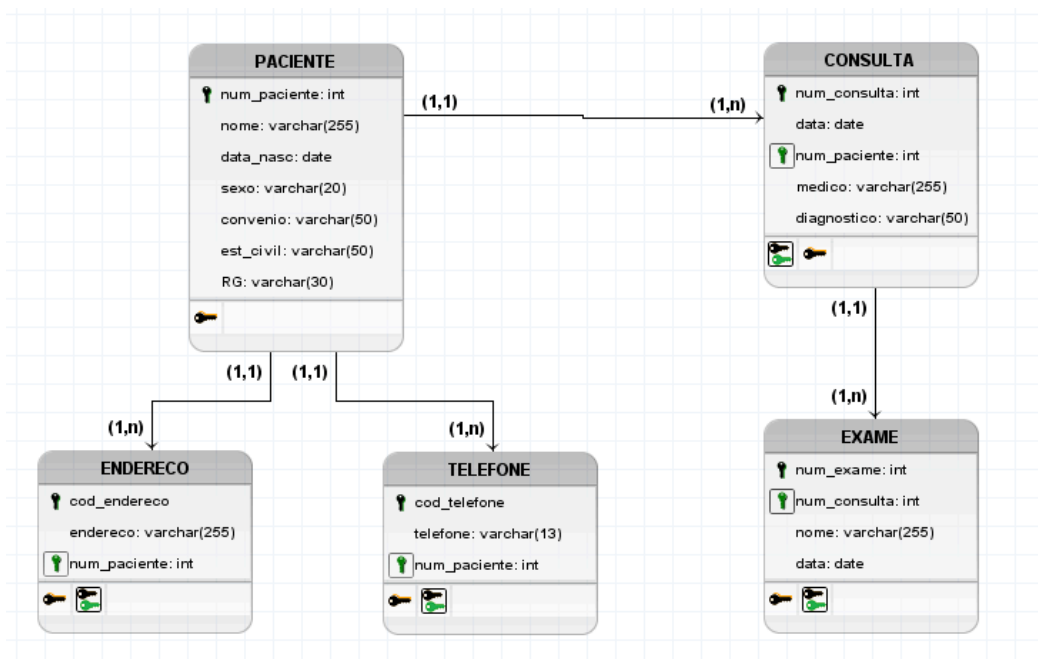


Figura 03 - Modelo Lógico

5. Desenvolvimento do Modelo Físico

Após a elaboração do modelo lógico, foi realizado o desenvolvimento do modelo físico do banco de dados utilizando SQL Server. Nessa etapa, as tabelas foram efetivamente implementadas no banco de dados, incluindo a definição dos tipos de dados, chaves primárias, chaves estrangeiras, restrições de integridade e utilização de atributos auto incrementáveis (IDENTITY). Foi o modelo físico que permitiu transformar a estrutura planejada nas etapas anteriores em um banco de dados funcional, preparado para integração com a aplicação desenvolvida em Python e Flask.

```
CREATE DATABASE Ficha_Medica
GO
USE Ficha_Medica

CREATE TABLE Paciente(
    num_paciente int primary key IDENTITY(0,1) NOT NULL,
```

```

    nome varchar(255) NOT NULL,
    data_nasc date,
    sexo varchar(20),
    convenio varchar(50) NOT NULL,
    est_civil varchar(50),
    RG varchar(30) unique NOT NULL
)

CREATE TABLE Endereco(
    cod_endereco int primary key IDENTITY(0,1) NOT NULL,
    endereco varchar(255) NOT NULL,
    fk_paciente int foreign key REFERENCES Paciente(num_paciente)
)

CREATE TABLE Telefone(
    cod_telefone int primary key IDENTITY(0,1) NOT NULL,
    telefone varchar(15) NOT NULL,
    fk_paciente int foreign key REFERENCES Paciente(num_paciente)
)

CREATE TABLE Consulta(
    num_consulta int primary key IDENTITY(0,1) NOT NULL,
    data_consulta date,
    medico varchar(255) NOT NULL,
    diagnostico varchar(255) NOT NULL,
    fk_paciente int foreign key REFERENCES Paciente(num_paciente)
)

CREATE TABLE Exame(
    num_exame int primary key IDENTITY(0,1) NOT NULL,
    nome varchar(255) NOT NULL,
    data_exame date,
    fk_consulta int foreign key REFERENCES Consulta(num_consulta)
)

```

6. API Flask em Python

Após a implementação do banco de dados, foi desenvolvida uma API utilizando Python e Flask com o objetivo de integrar o formulário HTML ao SQL Server. A API ficou responsável por receber as informações preenchidas pelo usuário, tratar os dados enviados e executar operações de inserção no banco de dados. Durante o desenvolvimento, foram implementadas funcionalidades como captura de dados via requisição HTTP 'POST', validação de registros já existentes (como RG por exemplo), utilização de chaves

estrangeiras para relacionamento entre tabelas e persistência das informações no banco de dados.

```
import pyodbc
from flask import Flask, jsonify, request, render_template

app = Flask(__name__)
app.secret_key = 'ArthurAlmeidaFichaAPI'

config = pyodbc.connect(
    'Driver={SQL Server};'
    'Server=DESKTOP-MLFI9DN;'
    'Database=Ficha_Medica;'
    'Trusted_Connection=yes;'
)

cursor = config.cursor()

@app.route('/')
def home():
    return render_template('index.html')

@app.route('/receber', methods=['POST'])
def receber():
    nome_paciente = request.form['nome_paciente']
    rg = request.form['RG']
    sexo = request.form['sexo']
    convenio = request.form['convenio']
    data_nasc = request.form['data_nasc']
    estado_civil = request.form['est_civil']

    endereco = request.form['endereco']
    telefone = request.form['telefone']

    data_consulta = request.form['data_consulta']
    medico = request.form['medico']
    diagnostico = request.form['diagnostico']

    exame = request.form['exame']
    data_exame = request.form['data_exame']

    cursor.execute("SELECT * FROM Paciente WHERE RG = ?", (rg,))
    if cursor.fetchone():
```

```
        return "Paciente já cadastrado!"

    cursor.execute("INSERT INTO Paciente(nome_paciente, data_nasc,
sexo, convenio, est_civil, RG) OUTPUT INSERTED.num_paciente VALUES (?,
?, ?, ?, ?, ?)", (nome_paciente, data_nasc, sexo, convenio,
estado_civil, rg))
    num_paciente = cursor.fetchone()[0]
    cursor.execute("INSERT INTO Endereco(endereco, fk_paciente)
VALUES(?, ?)", (endereco, num_paciente))
    cursor.execute("INSERT INTO Telefone(telefone, fk_paciente)
VALUES(?, ?)", (telefone, num_paciente))
    cursor.execute("INSERT INTO Consulta(data_consulta, medico,
diagnostico, fk_paciente) OUTPUT INSERTED.num_consulta VALUES(?, ?, ?,
?)", (data_consulta, medico, diagnostico, num_paciente))
    num_consulta = cursor.fetchone()[0]
    cursor.execute("INSERT INTO Exame(nome, data_exame, fk_consulta)
VALUES(?, ?, ?)", (exame, data_exame, num_consulta))
    config.commit()

    return 'Cadastro realizado com sucesso!'

app.run(debug=True)
```